

# Java Projekte bauen mit Maven



Eine kurze Einführung

**Dozent:** Prof. Dr. Michael Eichberg  
**Kontakt:** michael.eichberg@dhbw.de, Raum 149B  
**Version:** 1.0.5 [Themed]

---

**Folien:** <https://delors.github.io/prog-adv-java-projects/folien.de.rst.html>  
<https://delors.github.io/prog-adv-java-projects/folien.de.rst.html.pdf>  
**Fehler melden:** <https://github.com/Delors/delors.github.io/issues>  
**Kontrollfragen:** <https://delors.github.io/prog-adv-java-projects/kontrollfragen.de.rst.html>

# 1. Java Projekte bauen

# Ziele

0. **Management der Projektabhängigkeiten** (z. B. JUnit, Log4J, Hibernate, Spring, ....)
1. **Kompilieren** des Quellcodes
2. **Testen** des Quellcodes
3. **Paketieren** des Quellcodes (.jar oder .war Datei erzeugen inkl. *aller* Abhängigkeiten)
4. **Dokumentation** erstellen
5. **Reports** erstellen (z.B. Testabdeckung, Code-Qualität)
6. ... und vieles mehr, dass dann aber häufig Projektabhängig ist.

## ▲ Achtung!

Ein wichtiges Meta-Ziel ist es, das Bauen der Software zu automatisieren und zu vereinfachen und *stabile Builds zu gewährleisten*.

D. h. zwei Entwickler, die das selbe Projekt auf unterschiedlichen Rechnern mit initial ggf. unterschiedlichen Versionen installierter Werkzeuge und Bibliotheken bauen, sollten dennoch das selbe Ergebnis erhalten.

# Etablierte Build-Tools

- Ant<sup>[1]</sup>

- **Maven**

- Gradle

- sbt

- make (nicht spezifisch für Java)

## ⚠ Warnung

IDEs wie IntelliJ IDEA, Eclipse, Visual Studio Code oder NetBeans bieten ebenfalls „Build-Unterstützung“. Diese ist aber bestenfalls für kleine Ein-Entwickler-Projekte geeignet.

---

[1] Wurde in der Anfangsphase häufig verwendet. Heute nicht mehr.

# Projektstruktur

Konvention, die praktisch über alle Build-Tools und IDEs hinweg gilt<sup>[2]</sup>:

- Quellcode im Verzeichnis `src/main/java`
  - Testcode im Verzeichnis `src/test/java`
  - Ressourcen im Verzeichnis `src/main/resources`
  - Testressourcen im Verzeichnis `src/test/resources`
  - Konfigurationen und andere Ressourcen im Verzeichnis `src/main/resources`
  - gebaute Artefakte im Verzeichnis `target`
- 

[2] Andere Sprachen verwenden häufig ähnliche Strukturen. (Selbstverständlich, wird `java` dann durch den Namen der entsprechenden Sprache ersetzt.)

## 2. Grundlagen des Testens mit JUnit (Jupiter)

# JUnit - Einführung

- JUnit ist das Standard-Test-Framework für Java.
- Für das Schreiben von einfachen Tests ist das Modul *JUnit Jupiter* zu verwenden.
- Tests liegen (per Konvention) in `src/test/java` und werden *nicht* mit der Anwendung ausgeliefert.

# Aufbau einer Testklasse

```
1 import org.junit.jupiter.api.Test;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 class CalculatorTest { // Konvention: <Klasse>Test
5
6     @Test void testAddition() {
7         Calculator r = new Calculator();
8         assertEquals(5, r.addiere(2, 3));
9     }
10
11     @Test void testDivisionDurchNull() {
12         Calculator r = new Calculator();
13         assertThrows(
14             ArithmeticException.class,
15             () -> r.dividiere(1, 0)
16         );
17     }
18 }
```

## ▲ Achtung!

Testklassen, die mit dem Namen `Test` enden, werden von Buildtools automatisch erkannt. Es handelt sich um eine fest etablierte und weit verbreitete Konvention.

- Jede mit `@Test` annotierte Methode ist ein Testfall.
- Testmethoden müssen `void` zurückgeben und dürfen keine Parameter haben.
- Es wird für **jeden** Testfall ein **neues** Objekt der Testklasse erzeugt; Tests sind daher voneinander isoliert.



# Die wichtigsten Assertions

Alle Methoden befinden sich in `org.junit.jupiter.api.Assertions`:

| Methode                                            | Prüft                                |
|----------------------------------------------------|--------------------------------------|
| <code>assertEquals(expected, actual)</code>        | <code>expected.equals(actual)</code> |
| <code>assertNotEquals(a, b)</code>                 | <code>!a.equals(b)</code>            |
| <code>assertTrue(condition)</code>                 | <code>condition == true</code>       |
| <code>assertFalse(condition)</code>                | <code>condition == false</code>      |
| <code>assertNull(object)</code>                    | <code>object == null</code>          |
| <code>assertNotNull(object)</code>                 | <code>object != null</code>          |
| <code>assertThrows(Exc.class, () -&gt; ...)</code> | Lambda wirft erwartete Exception     |
| <code>assertArrayEquals(exp, act)</code>           | Arrays elementweise gleich           |

## ※ Hinweis

Jede Assertion akzeptiert als *letzten* Parameter eine optionale Fehlermeldung (`String`), die bei einem Fehlschlag ausgegeben wird:

```
assertEquals(5, r.addiere(2, 3), "2+3 sollte 5 sein");
```

# Setup und Teardown

```
1 import org.junit.jupiter.api.*;
2 import static org.junit.jupiter.api.Assertions.*;
3
4 class ListTest {
5
6     private List list;
7
8     @BeforeEach // vor JEDEM einzelnen Test
9     void setUp() { list = new List(); }
10
11     @AfterEach // nach JEDEM einzelnen Test
12     void tearDown() { /* z.B. Ressourcen freigeben */ }
13
14     @Test
15     void testIsEmpty() { assertTrue(list.isEmpty()); }
16
17     @Test
18     void testAddElement() { list.add(42); assertEquals(1, list.size()); }
19 }
```

---

`@BeforeEach` und `@AfterEach` werden vor bzw. nach *jedem* Testfall ausgeführt und eignen sich ideal, um einen definierten Ausgangszustand herzustellen.

Daneben gibt es `@BeforeAll` und `@AfterAll` (müssen `static` sein), die nur *einmal* pro Testklasse aufgerufen werden.

# Typische Vorgehensweise beim Testen

## Arrange – Act – Assert (AAA-Muster)

1. **Arrange** – Testdaten und Objekte vorbereiten.
2. **Act** – Die zu testende Methode aufrufen.
3. **Assert** – Das Ergebnis mit dem erwarteten Wert vergleichen.

```
1 @Test
2 void testAbsoluterBetrag() {
3     /* Arrange:*/ int eingabe = -7;
4     /* Act:    */ int ergebnis = Math.abs(eingabe);
5     /* Assert: */ assertEquals(7, ergebnis);
6 }
```

### ⌘ Hinweis

**Ein** Test sollte genau **einen** Aspekt prüfen. Besser mehrere kleine Tests als ein großer.

## 3. Maven

<https://maven.apache.org>

# Aufsetzen eines Projekts mittels *Scaffolding*

Maven ermöglicht es, den Rumpf für ein Java-Projekt mit einer einfachen Befehlszeile zu erstellen:

```
mvn archetype:generate \  
  -DgroupId=com.mycompany.app \  
  -DartifactId=my-app \  
  -DarchetypeArtifactId=maven-archetype-quickstart \  
  -DarchetypeVersion=1.5 \  
  -DinteractiveMode=false
```

Dies erzeugt eine initiale Build-Konfiguration für ein einfaches Java-Projekt und erzeugt die Projektstruktur.<sup>[3]</sup>

---

Die `groupId` folgt dabei den selben Konventionen wie Java-Packages. Die `artifactId` ist der Name des Projekts.

---

[3] Es gibt eine Vielzahl von Archetypen, die unterschiedliche Projektstrukturen erzeugen und für unterschiedliche Anwendungsfälle optimiert sind.

# Maven - Build Phasen

■ Eine Phase ist ein Schritt im Build-Lebenszyklus. Die *ersten* Phasen des Standardlebenszyklus sind:

1. `validate`
2. `generate-sources`
3. `process-sources`
4. `generate-resources`
5. `process-resources`
6. `compile`

■ Wenn eine Phase angegeben wird, dann werden alle vorherigen Phasen ausgeführt. Zum Beispiel führt `mvn compile` alle genannten Phasen in obiger Reihenfolge aus.

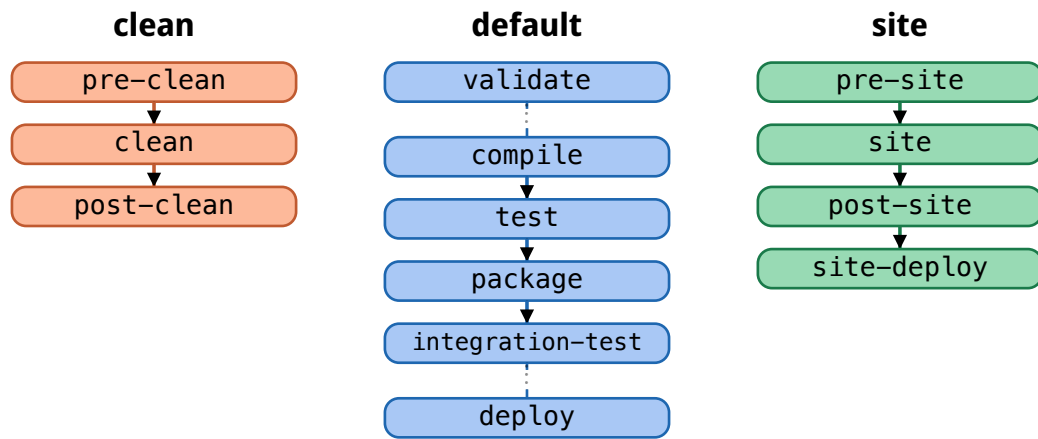
## die wichtigsten Phasen des Standardlebenszyklus

|                                |                                                                                                                               |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <code>validate:</code>         | überprüfen, ob das Projekt korrekt konfiguriert ist                                                                           |
| <code>compile:</code>          | kompilieren des Quellcodes des Projekts                                                                                       |
| <code>test:</code>             | testet den kompilierten Quellcode mit einem geeigneten Unit-Testing-Framework.                                                |
| <code>package:</code>          | den kompilierten Code in ein verteilbares Format, z. B. ein JAR, verpacken.                                                   |
| <code>integration-test:</code> | Verarbeitet das Paket und stellt es, wenn nötig, in einer Umgebung bereit, in der Integrationstests ausgeführt werden können. |
| <code>deploy:</code>           | bereitstellen in einer Integrations- oder Release-Umgebung                                                                    |

## Spezialisierte Lebenszyklen (mit eigenen Phasen)

|                     |                                                                                                                                                               |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>clean:</code> | bereinigt Artefakte, die von früheren Builds erzeugt wurden.<br>Phasen: <code>pre-clean</code> , <code>clean</code> , <code>post-clean</code>                 |
| <code>site:</code>  | generiert eine Site-Dokumentation für dieses Projekt<br>Phasen: <code>pre-site</code> , <code>site</code> , <code>post-site</code> , <code>site-deploy</code> |

# Maven Lebenszyklen und Phasen - Übersicht



# Beispiel Build-Konfiguration für ein Java Projekt

Code der Anwendung (in `src/main/java/<package>/<class>.java`)

```
1 package de.dhlbw;
2
3 /**
4  * Implements the main method to greet a user by name.
5  */
6 public class HelloYou {
7
8     public static void main(String[] args) {
9         if (args.length == 0) {
10             System.out.println("Usage: java HelloYou <name>");
11             return;
12         }
13         System.out.println("Hello " + args[0] + "!");
14     }
15 }
```

TestCode (in `src/test/java/<package>/<class>.java`)

(Herausforderung: Testing `System.out`)

## Header

```
10 public class HelloYouTest {
11
12     // Let's redirect System.out to capture the output!
13     private final PrintStream defaultOut = System.out;
14     private final ByteArrayOutputStream testOut = new ByteArrayOutputStream();
15 }
```

## Setup

```
15 @BeforeEach
16 public void setUpOutputStream() {
17     final var out = new PrintStream(testOut);
18     System.setOut(out);
19 }
20
21 @AfterEach
22 public void resetSystemOut() {
23     System.setOut(defaultOut);
24 }
```

## Eigentliche Tests

```
28 @Test
29 void testMainNoArgs() {
30     HelloYou.main(new String[0]);
31     assertEquals("Usage: java HelloYou <name>\n", testOut.toString());
32 }
33
34 @Test
35 void testMainArg() {
36     HelloYou.main(new String[] { "Bob" });
37     assertEquals("Hello Bob!\n", testOut.toString());
38 }
```











